

M	T	W	T	F	S	S
			1	2	3	4
5	6	7	8	9	10	11
12	13	14	15	16	17	18
19	20	21	22	23	24	25
26	27	28	29			

Functional Dependencies

→ FD is a constraint b/w 2 set of attributes in a relation from a DB

A FD $A \rightarrow B$ means

if you know A, you can uniquely determine B

OR

The attribute B is functionally dependent on attribute A

Example:- (College System DB)

Roll No

Name

Branch

Email

1201

Vaibhav

Mechanical

vaibhav@gmail.com

1202

Arum

Civil

arum@xyz.edu

!

!

!

!

Roll No $\rightarrow$ Name, Branch, Email

But Name $\rightarrow$ Roll No. X (Because duplicates possible)

11 SUNDAY

Types of FD

① Trivial FD : $A \rightarrow B$ is trivial if $B \subseteq A$

ex:- $\{\text{Roll No, Name}\} \rightarrow \text{Name}$

M	T	W	T	F	S	S
				1	2	3
4	5	6	7	8	9	10
11	12	13	14	15	16	17
18	19	20	21	22	23	24
25	26	27	28	29	30	31

2024

FEBRUARY • MONDAY

12

② Non-Trivial FD: $A \rightarrow B$, $B \not\subseteq A$

Ex:- Roll No $\rightarrow$ Name

③ Fully FD: $A \rightarrow B$, and removing any attribute from A breaks the dependency

Ex:- {Stu.Id, Course Id} $\rightarrow$ Grade

④ Partial FD: $A \rightarrow B$ where an attribute in B is only partially dependent on A

Ex:- StuId $\rightarrow$ Stu. Name but to identify grade CourseId is also required along with StuId

⑤ Transitive FD: $A \rightarrow B$ and $B \rightarrow C \Rightarrow A \rightarrow C$

Ex:- Roll No. $\rightarrow$ MentorId and MentorId $\rightarrow$ MentorName
 $\Rightarrow$ Roll No. $\rightarrow$ Mentor Name

Armstrong's Axioms (FD Inference rules)

① Reflexivity: if $Y \subseteq X$ then $X \rightarrow Y$

{StudentId, Name} $\rightarrow$ Name

② Augmentation: if $X \rightarrow Y$ then $XZ \rightarrow YZ$ for any Z

Roll no. $\rightarrow$ Name

then {Roll no., Branch} $\rightarrow$ {Name, Branch}

Success is the combination of both vision and accepting the challenge

M	T	W	T	F	S	S
			1	2	3	4
5	6	7	8	9	10	11
12	13	14	15	16	17	18
19	20	21	22	23	24	25
26	27	28	29			

③ Transitivity: if $X \rightarrow Y$ and $Y \rightarrow Z$ then $X \rightarrow Z$
 Roll No $\rightarrow$ Mentor Id and Mentor Id $\rightarrow$ Mentor Name
 then Roll No $\rightarrow$ Mentor Name

④ Union: if $X \rightarrow Y$ and $X \rightarrow Z$ then $X \rightarrow YZ$
 Roll No $\rightarrow$ Name and Roll No $\rightarrow$ Email
 then Roll No $\rightarrow$ {Name, Email}

⑤ Decomposition: if $X \rightarrow YZ$ then $X \rightarrow Y$ and $X \rightarrow Z$

Ex: Vice-versa of Union

⑥ Pseudotransitivity: if $X \rightarrow Y$ and $YZ \rightarrow W$ then $XZ \rightarrow W$

Roll No $\rightarrow$ Branch and Branch, Semester $\rightarrow$ Mentor
 $\Rightarrow$ {Roll No, semester} $\rightarrow$ Mentor

Types of Keys

① Super Key: Any set of attributes that can uniquely identify a record in an entity set.

Ex: For entity Student

Super keys: {Roll No}, {Roll No, Name}, {Roll No, Email}

$\hookrightarrow$ Can have extra fields also

M	T	W	T	F	S	S
				1	2	3
4	5	6	7	8	9	10
11	12	13	14	15	16	17
18	19	20	21	22	23	24
25	26	27	28	29	30	31

② Candidate Key :- Minimal Super key (no extra attributes)

If {Roll No} alone can identify a student, it is a candidate key

{Roll No, Name} → X (Not a candidate key)

An entity can have multiple candidate keys

{Stu Id}, {Stu Aadhar}

③ Primary Key :- Chosen candidate key which acts as main identifier

should be:- unique, not null, stable (won't change over time)

④ Alternate Key :- All the candidate keys other than Primary key

⑤ Foreign Key :- Keys from one table that is used to point to the primary key of other table.

Student entity has a College Id which is primary key in College entity
Foreign Key

⑥ Composite Key :- Combination of two or more candidate keys

For Course Enrollment entity :- {StudentId, CourseId}

M	T	W	T	F	S	S
			1	2	3	4
5	6	7	8	9	10	11
12	13	14	15	16	17	18
19	20	21	22	23	24	25
26	27	28	29			

Finding Candidate Keys and Super Keys using FD

Ques- Given a relation $R(A, B, C, D)$ and FDs

$$A \rightarrow B, A \rightarrow C, B, C \rightarrow D$$

Solⁿ

Super Keys $\{A, B, C, D\}$

$\{A, C, D\}$

$\{A, B, C\}$

$\{A, D\}$

$\{A\}$

← Candidate Key

Canonical Cover of FD

↪ The canonical cover (minimal cover) of a set of FDs is a simplified set of dependencies that is logically equivalent to the original set but has the following properties:-

- ① Each FD has only a single attribute on the right-hand side.
- ② The left hand side of each FD is minimal.
- ③ NO functional dependency can be removed w/o changing the closure of the set. (NO Redundant FDs).

(A^+) → Set of all attributes that can be determined from A using FDs

M	T	W	T	F	S	S
				1	2	3
4	5	6	7	8	9	10
11	12	13	14	15	16	17
18	19	20	21	22	23	24
25	26	27	28	29	30	31

2024

FEBRUARY • FRIDAY

16

Anomalies

→ Problems that arise due to poor table structure, usually because of data redundancy.

Types of Anomalies① Insertion Anomaly

Occurs when certain attributes cannot be inserted into the DB w/o the presence of other attributes.

ex:- In Student-Course table if you want to add new course 'DBMS' which no one has enrolled in yet but you can't insert because student name is also required forcing you to insert fake student just to store course

② Update Anomaly

Occurs when one or more instances of duplicated data need to be updated, leading to data inconsistency.

③ Deletion Anomaly

Occurs when deleting certain data inadvertently causes loss of other valuable data.

stu Name	Course Name	Marks
Vaibhan	DBMS	95

If Vaibhan drops DBMS & delete this row, then we also lose the info that DBMS course exists

Full effort is full victory. Satisfaction lies in effort

Normalization

Process of organizing data in a DB to minimize redundancy & avoid anomalies. The goal of normalization is to divide large tables into smaller, more manageable pieces while maintaining relationships b/w the data.

Normalization Types (Normal Forms)

① 1NF

The values in each column of a table are atomic (no multi-valued attributes allowed).

Ex:-	Genre	Name	Movie	
	Marvel	Kevin Feige	The Avengers, Captain America	} Not in 1NF
	DC	Zack Snyder	Batman V Superman, Suicide Squad	

Convert
to 1NF

Decompose the table
into atomic elements

	Genre	Name	Movie
18 SUNDAY	Marvel	Kevin Feige	The Avengers
	Marvel	Kevin Feige	Captain America
	DC	Zack Snyder	Batman V Superman
	DC	Zack Snyder	Suicide Squad

M	T	W	T	F	S	S
				1	2	3
4	5	6	7	8	9	10
11	12	13	14	15	16	17
18	19	20	21	22	23	24
25	26	27	28	29	30	31

2024

FEBRUARY • MONDAY

19

② 2NF

A relation is in 2NF if it is in 1NF and all non-key attributes are fully functionally dependent on the primary key.

Example

StuId	CourseId	Stu Name	Course Name	Grade
101	DBMS	Vaibhau	CS-DBMS	A
101	OS	Vaibhau	CS-OS	B
102	DBMS	Cheryl	CS-DBMS	A

→ Candidate key = {StuId, CourseId}

- Grade depends on both StuId & CourseId
 - Stu Name does not need CourseId
 - Course Name does not need StuId
- Partially dependent on Candidate key

Problem: ① Stu Name is repeated unnecessarily for every course
② Course Name is repeated for every student

This leads to:- (i) Redundancy

(ii) Update Anomalies

(What if you spell 'DBMS' differently once?)

Inconsistency

Now, converting to 2NF

Alone we can do so little; together we can do so much

20

2024

TUESDAY • FEBRUARY

FEBRUARY 2024

M	T	W	T	F	S	S
			1	2	3	4
5	6	7	8	9	10	11
12	13	14	15	16	17	18
19	20	21	22	23	24	25
26	27	28	29			

Table 1	<u>Student</u>	<u>StuId</u>	<u>StuName</u>
9		101	Vaibhav
		102	Cheryl

Table 2	<u>Course</u>	<u>CourseId</u>	<u>CourseName</u>
11		DBMS	CS-DBMS
		OS	CS-OS

Table 3	<u>Enrollment</u>	<u>StuId</u>	<u>CourseId</u>	<u>Grade</u>
1		101	DBMS	A
		101	OS	B
2		102	DBMS	A

③ 3NF

4 If it is in 2NF and no non-Key attribute is transitively dependent on the primary key

5 Examples-

<u>StuId</u>	<u>Comp. Name</u>	<u>Placed Status</u>	<u>HR Name</u>	<u>HR Contact</u>
7 101	Google	Selected	Raju	9876543210
102	Amazon	Rejected	Bunty	9123456780
103	Google	Pending	Raju	9876543210

↳ Candidate Key = {StuId, Company Name}

(Because one student can apply to multiple companies)

M	T	W	T	F	S	S
				1	2	3
4	5	6	7	8	9	10
11	12	13	14	15	16	17
18	19	20	21	22	23	24
25	26	27	28	29	30	31

2024

FEBRUARY • WEDNESDAY

21

9 FDs :- { StuId, Comp.Name } $\rightarrow$ Placed Status

Comp.Name $\rightarrow$ HR Name $\leftarrow$ Problem

HR Name $\rightarrow$ HR Contact $\leftarrow$ (Transitive Dependency)

11 Now, Converting to 3NF

Table 1	Placement Status	StuId	Comp. Name	Placed Status
		101	Google	Selected
		102	Amazon	Rejected
		103	Google	Pending

Table 2	Company HR	Comp. Name	HR Name
		Google	Raju
		Amazon	Bimty

Table 3	HR Details	HR Name	HR Contact
		Raju	9876543210
		Bimty	9123456780

④ BCNF (Boyce-Codd Normal Form)

$\rightarrow$ It is in 3NF & for every FD $X \rightarrow Y$, X must be a super key.

Ex:-

RollNo.	Subject	Faculty	Lab Room
CS1001	DBMS	Dr. Raj	L1
CS1002	CN	Dr. Meera	L2
CS1001	CN	Dr. Meera	L2

A good plan violently executed now is better than a perfect plan next week

Table is in 1NF & 2NF but there is a transitive dependency:-

Subject $\rightarrow$ Faculty
Faculty $\rightarrow$ Lab Room

But subject is not a super key $\rightarrow$ BCNF violated

Fixing BCNF:-

Table 1 Student Submissions

Roll No	Subject
CS 1001	DBMS
CS 1002	CN
CS 1001	CN

Table 2 Subjects Faculty

Subject	Faculty
DBMS	Dr. Raj
CN	Dr. Meera

Table 3 Faculty Lab

Faculty	Lab Room
Dr. Raj	L1
Dr. Meera	L2

⑤ 4NF

$\hookrightarrow$ If the table is in BCNF and it has no multi-valued dependencies (no attribute should have multiple independent values for the same key)

M	T	W	T	F	S	S
				1	2	3
4	5	6	7	8	9	10
11	12	13	14	15	16	17
18	19	20	21	22	23	24
25	26	27	28	29	30	31

2024

FEBRUARY • FRIDAY

23

Example:-

Student	Society	Language
Vaibhan	C2C Club	C++
Vaibhan	C2C Club	Python
Vaibhan	Robotics Club	C++
Vaibhan	Robotics Club	Python

Problem ??

These were just 2 facts that Vaibhan is in 2 societies & Vaibhan knows 2 languages but we form a table $2 \times 2 = 4$ rows

Language & Society are not linked, just the different associations of Vaibhan

Student is the key & it determines 2 independent multi-values } 4NF violation

Fixing 4NF 1:-Table 1 Student Society

Student	Society
Vaibhan	C2C Club
Vaibhan	Robotics Club

Table 2 Student Language

Student	Language
Vaibhan	C++
Vaibhan	Python

The Strength of a human character are defined by its moral reserves

⑥ 5NF

↳ if it is in 4NF & it cannot be broken into smaller tables without data loss even using joins.

⑦ Example:-

⑧ Scenario:- A system where multiple students work on multiple projects and each project is guided by multiple prof.

Table:- Project Allotment

Student	Project	Prof.
Vaibhav	DBMS Tool	Dr. Raj
Vaibhav	DBMS Visual	Dr. Meera
Cheryl	DBMS Tool	Dr. Raj
Cheryl	DBMS Visual	Dr. Meera

Now suppose this data is not explicitly stored, but you derive this table by joining:-

① Student Project ② Project Prof. ③ Student Prof.

But, while joining, we might get false combinations like
25 SUNDAY Vaibhav working with Dr. Meera on DBMS tool

↳ Join Dependency Violation

So, if breaking a table causes wrong combinations during join → 5NF is violated

Relational Decomposition

Process of breaking down a larger relation (table) into smaller relations that preserve the original data but reduce redundancy and anomalies.

① Lossless (Non-lossy) Decomposition

Ensures no information is lost when the decomposed relations are joined back together.

Condition:- A decomposition of relation R into R_1 & R_2 is lossless if the common attribute b/w R_1 & R_2 form a super key in atleast one of the decomposed relations.

$R_1 \cap R_2 \rightarrow R_1$ or R_2 (Common attributes should determine one full table)

Example:- Student Full

Roll No.	Name	Hostel
101	Vaibhav	H1
102	Cheryl	H2

FD:- $\text{Roll No.} \rightarrow \{\text{Name, Hostel}\}$

Decompose into → $\left. \begin{array}{l} \text{Student Basic (Roll No, Name)} \\ \text{Hostel Info (Roll No, Hostel)} \end{array} \right\} \begin{array}{l} \text{common attribute} \\ \underline{\text{Roll No.}} \end{array}$

Lossless Decomposition because common attribute is key in both

② Dependency Preserving Decomposition

→ After decomposition, we can still enforce all FDs using the decomposed tables (w/o needing to join them again)

Condition 1:- If F is the set of FDs in the original relation R , and $F_1, F_2, \dots, F_n$ are the sets of FDs in the decomposed relations, then $F_1 \cup F_2 \cup \dots \cup F_n = F$

Example 1

<u>Student</u>	Roll No	Name	Dept	Mentor
	101	Vaibhav	CSE	Raj
	102	Cheryl	CSE	Raj Raj
	103	Myra	Civil	Meera

FDs Rollno $\rightarrow$ Name, Dept, Mentor

Dept $\rightarrow$ Mentor

Decompose into R_1 (Rollno, Name, Dept)
 R_2 (Dept, Mentor)

All FDs are preserved w/o joining $\rightarrow$ Dependency Preserving

Lossless join is typically prioritized over dependency preservation as data integrity is more important than maintaining all FDs directly.

M	T	W	T	F	S	S
				1	2	3
4	5	6	7	8	9	10
11	12	13	14	15	16	17
18	19	20	21	22	23	24
25	26	27	28	29	30	31

2024

FEBRUARY • WEDNESDAY

28

Denormalization

Process of intentionally introducing redundancy into a DB by combining related tables.

Why Denormalize ??

While normalization is essential for eliminating redundancy and anomalies, it often results in a larger number of tables, which can increase the complexity of DB operations. Specifically, multiple joins b/w tables during read operations can slow down query performance, that's why denormalization is used.

When to Denormalize ??

- ① Read-Heavy Systems
- ② When joins are expensive
- ③ Caching

FEBRUARY 2024						
M	T	W	T	F	S	S
			1	2	3	4
5	6	7	8	9	10	11
12	13	14	15	16	17	18
19	20	21	22	23	24	25
26	27	28	29			

Transactions

Sequence of one or more operations (such as read, write or update) executed as a single logical unit.

Example:- Money Transfer: Debit from Account A
Credit to Account B

If either step fails, the entire transaction must roll back to maintain consistency.

Transaction states

- ① Active → The transaction is in progress, performing its operations.
- ② Failed → An error (system crash, validation failure) prevents the transaction from proceeding.
- ③ Aborted → The transaction is rolled back and any changes made are undone.
- ④ Partially Committed → All operations have been executed but the transaction is not yet finalized.
- ⑤ Committed → The transaction completes successfully and all changes are permanently saved to the DB.

March

03

ACTION PLAN

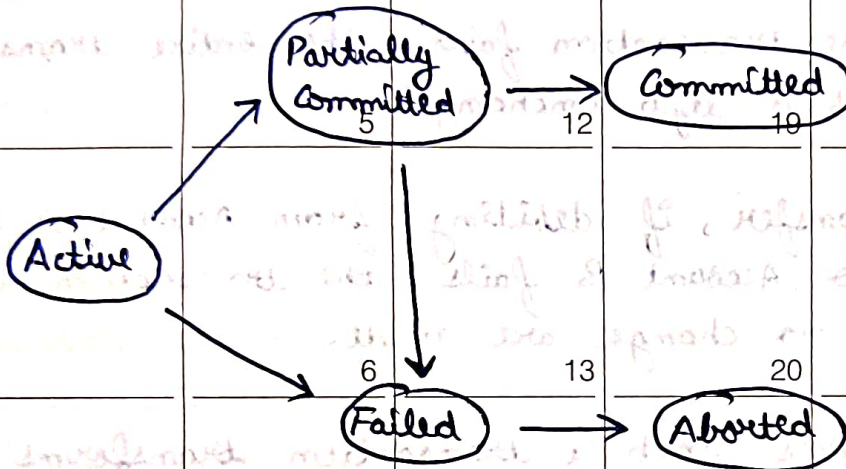
MONTH IN VIEW

2024

APRIL '24

M	T	W	T	F	S	S
1	2	3	4	5	6	7
8	9	10	11	12	13	14
15	16	17	18	19	20	21
22	23	24	25	26	27	28
29	30					

MONDAY		4	11	18	25
TUESDAY		5	12	19	26
WEDNESDAY		6	13	20	27
THURSDAY		7	14	21	28
FRIDAY		8	15	22	29
SATURDAY		9	16	23	30
SUNDAY		10	17	24	31



No one
can go back
and change
a bad
beginning;
But anyone
can start now
and create
a successful
ending.

MARCH

APRIL

MARCH 2024						
M	T	W	T	F	S	S
4	5	6	7	1	2	3
11	12	13	14	8	9	10
18	19	20	21	15	16	17
25	26	27	28	22	23	24
			29	29	30	31

ACID Properties

Ensures that DB transactions are processed reliably even in cases of failure, concurrency or system crashes.

① Atomicity → Ensures that a transaction is all or nothing; if any part of the transaction fails, the entire transaction fails and the DB is left unchanged.

Ex In a bank transfer, if debiting from Account A succeeds but crediting to Account B fails, the transaction is rolled back and no changes are made.

② Consistency → Ensures that a transaction transforms the DB from one consistent state to another consistent state, ensuring integrity constraints are not violated.

Ex After a transfer, the sum of the balances of both accounts should remain the same.

③ Isolation → Multiple transactions running at the same time do not impact each other's execution.

Ex If two or more people try to buy the last copy of a product, just one of them will succeed.

④ Durability → Ensures that once a transaction is committed, the changes are permanent and persist even in case of a system crash.

Unless commitment is made, there are only promises and hopes; but no plans

M	T	W	T	F	S	S
1	2	3	4	5	6	7
8	9	10	11	12	13	14
15	16	17	18	19	20	21
22	23	24	25	26	27	28
29	30					

2024

MARCH • SATURDAY

02

Concurrent Execution

When multiple transactions are executed at the same time (overlapping in time), that's called concurrent execution.

They may not run at the exact same moment, but their operations are interleaved (mixed) - so it feels like parallel execution.

Why needed ??

Real-world systems have 100s or 1000s of users accessing DB together.

Example:- you and your friend both filling the same google form for registration at the same time.

Executing one transaction at a time (serially) is :-

- Very slow
- Wastes system resources (CPU, IO)

So, we allow concurrent execution for :-

- Better CPU utilization
- Better system throughput
- Faster response time

SUNDAY 03

Problems with Concurrent Execution ??

Let's say Vaibhav and Cheryl are booking the last seat in a Coding Bootcamp. Both press Book at the same time.

The aim of the wise is not to secure pleasure, but to avoid pain.

MARCH 2024						
M	T	W	T	F	S	S
				1	2	3
4	5	6	7	8	9	10
11	12	13	14	15	16	17
18	19	20	21	22	23	24
25	26	27	28	29	30	31

If DB doesn't handle this properly, both may get confirmation.

This is a Concurrency Problem

Common Problems in Concurrent Execution

① Dirty Reads - Reading uncommitted data of another transaction

Ex- Vaibhav updates a record, but Cheryl reads it before Vaibhav commits, if it rolls back, Cheryl has read incorrect data.

② Lost Updates - Two transactions overwrite each other's updates

Ex- T_1 updates the balance to 100 but T_2 concurrently updated it to 50, overwriting T_1 's update.

③ Unrepeatable Reads - A transaction reads the same data twice but gets different values because another transaction modified the data b/w the reads.

④ Phantom Reads - A transaction reads a set of rows that match a condition, but another transaction inserts or deletes rows making the set different on subsequent reads.

M	T	W	T	F	S	S
1	2	3	4	5	6	7
8	9	10	11	12	13	14
15	16	17	18	19	20	21
22	23	24	25	26	27	28
29	30					

2024

MARCH • TUESDAY

05

Serializability

Ensures that concurrent execution of transactions gives the same result as if the transactions were run one after another (serially).

It doesn't mean we actually run them one by one. It just means final result is same as some serial order even if they are interleaved.

Example:-

Vaibhav and Cheryl are using 2 ATMs at the same time to withdraw from the same account (say ₹ 1000)

Now, let's say Vaibhav withdraws ₹ 200 & Cheryl ₹ 500

If both, happen together (concurrent), final balance should be ₹ 300 either way.

But if system messes up interleaving

- Both read ₹ 1000
- Both subtract their amounts
- Both write updated value

Result = ₹ 500 or ₹ 800 (Violation of serializability)

Types of serializability

- Ordinary people while working together can perform extraordinary feats

M	T	W	T	F	S	S
				1	2	3
4	5	6	7	8	9	10
11	12	13	14	15	16	17
18	19	20	21	22	23	24
25	26	27	28	29	30	31

① Conflict Serializability

A schedule is conflict serializable if it can be transformed into a serial schedule by swapping non-conflicting operations.

② View Serializability

A schedule is view serializable if the transaction produce the same output as some serial execution.

Two operations conflict if:

- (i) They belong to different transactions
- (ii) Access the same data item
- (iii) Atleast one is a write

M	T	W	T	F	S	S
1	2	3	4	5	6	7
8	9	10	11	12	13	14
15	16	17	18	19	20	21
22	23	24	25	26	27	28
29	30					

2024

MARCH • THURSDAY

07

Recoverability

Ensures that if one transaction depends on another, it should only commit if the one it depends on also commits. Otherwise, the DB should be able to recover safely if something fails.

Example:- Vaibhav writes the code & Cheryl writes the documentation based on Vaibhav's code but Vaibhav never submitted the code but Cheryl submits the documentation. That's unfair because documentation is now useless.

Types of Recoverability

- ① Recoverable - T_2 reads from $T_1 \Rightarrow T_2$ commits after T_1
- ② Cascadeless - T_2 reads from only committed transactions
- ③ strict - T_2 reads/writes only after T_1 commits or aborts
- ④ Non-recoverable - T_2 commits before T_1 (from whom it read)
- ⑤ Cascading Aborts - T_1 aborts $\rightarrow$ all who read from T_1 must abort too.

M	T	W	T	F	S	S
				1	2	3
4	5	6	7	8	9	10
11	12	13	14	15	16	17
18	19	20	21	22	23	24
25	26	27	28	29	30	31

Concurrency Control Protocols

① Lock Based Protocols

Prevent conflicting operations from executing concurrently by using locks.

Types of Locks

(i) Shared Lock (S) - For Reading (multiple transactions can share)

(ii) Exclusive Lock (X) - For Writing (only one transaction allowed)

Two-Phase Locking Protocol (2PL)

Transaction works in two phases:-

(i) Growing Phase - A transaction acquires locks but cannot release any.

(ii) Shrinking Phase - A transaction releases locks but cannot acquire new ones.

Example:-

T_1 : Lock-S(A) $\rightarrow$ Read(A) $\rightarrow$ Lock-X(B) $\rightarrow$ Write(B) $\rightarrow$ Unlock(A) $\rightarrow$ Unlock(B)

T_2 : Lock-S(B) $\rightarrow$ Read(B)

If T_2 tries to read B before T_1 releases X-lock on B, it will be forced to wait $\rightarrow$ Conflict Avoided

Drawback of 2PL :- Can lead to Deadlocks

Great organizations demand a high level of commitment

M	T	W	T	F	S	S
1	2	3	4	5	6	7
8	9	10	11	12	13	14
15	16	17	18	19	20	21
22	23	24	25	26	27	28
29	30					

2024

MARCH • SATURDAY

09

② Timestamp Ordering Protocol (TO Protocol)

Every transaction gets a unique timestamp, and all operations must happen in timestamp order to avoid conflicts.

Each data item maintains:-

- (i) read- $TS(x) \rightarrow$ Max. timestamp of Transaction that read x
- (ii) write- $TS(x) \rightarrow$ Max. timestamp of Transaction that wrote x

Rules for Read/Write Operations

① Read(x) by T_i (timestamp $TS(T_i)$)

- Allowed if $TS(T_i) \geq \text{write-}TS(x)$

② Write(x) by T_i

- Allowed if $TS(T_i) \geq \text{read-}TS(x)$ and $TS(T_i) \geq \text{write-}TS(x)$

Benefits:- (i) No need for Locks

(ii) Avoid Deadlocks

Drawbacks:- (i) Can lead to Starvation

(ii) Too many abort-restarts, especially in high concurrency situations

SUNDAY 10

M	T	W	T	F	S	S
				1	2	3
4	5	6	7	8	9	10
11	12	13	14	15	16	17
18	19	20	21	22	23	24
25	26	27	28	29	30	31

③ Validation Based Protocol (Optimistic Concurrency Control)

Conflict check is done only at the end, not while running,

Every Transaction T_i goes through 3 phases:-

i) Read Phase

T_i reads data from DB and performs all calculations in local memory (No changes made to actual DB yet)

ii) Validation Phase

Before writing anything to DB, T_i checks for conflicts with other ongoing/committed Transactions.

iii) Write Phase

If no conflicts found, then apply changes to DB otherwise abort & retry

When it is Best ??

- i) System having fewer Transactions
- ii) Rare Conflicts
- iii) Analytics systems (Read-heavy operations)

Advantages:-

- i) No blocking of transactions during execution
- ii) No Deadlocks
- iii) No waiting

Drawbacks:-

- i) Too many aborts
- ii) Bad for write-heavy systems
- iii) Starvation